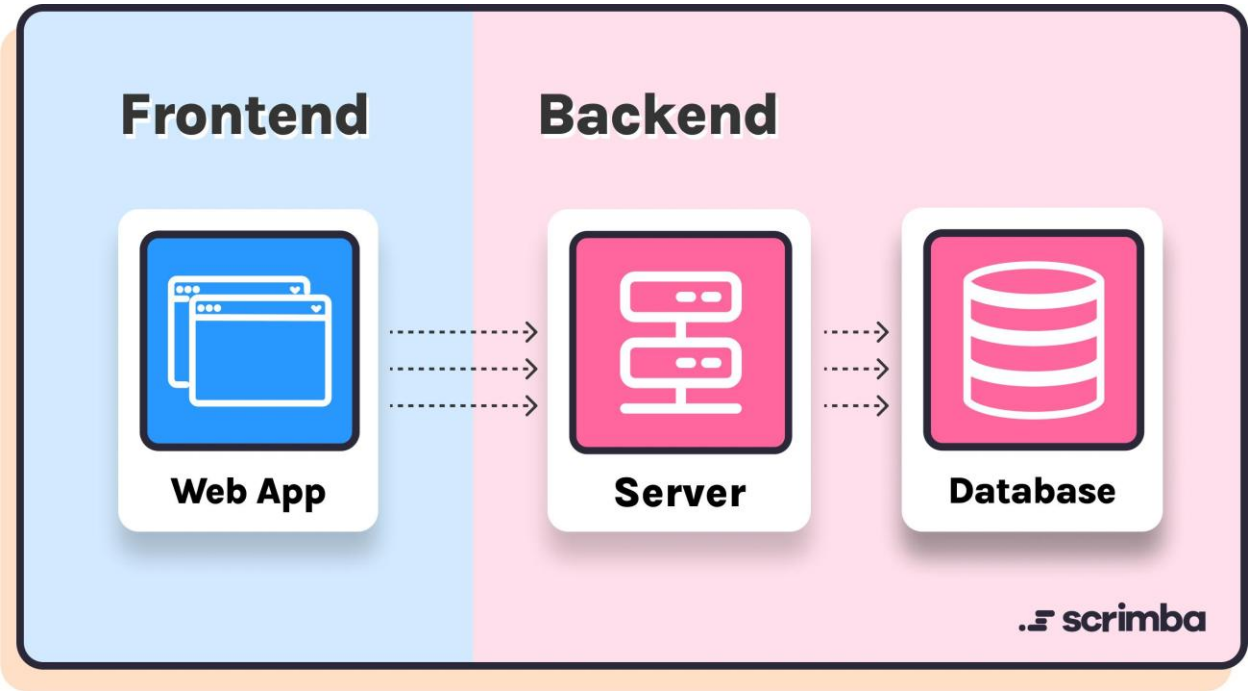
1] What is Full Stack Development?

Full Stack Development is the process of developing both the frontend and backend of a web application and connecting them to a database when needed.

A full stack developer can work on different parts of an application, from the user interface to server-side logic and data storage.



Main parts of Full Stack Development

1. Frontend

The part of the application that users see and interact with.

Example: buttons, forms, menus, and web pages.

2. Backend

The part that processes requests, applies business rules, and communicates with the database.

Example: checking login details or calculating an order total.

3. Database

Stores application data so it can be retrieved, updated, and managed.

Example: user details, products, and orders.

Example

Imagine that you log in to a website.

1. The frontend displays the login form.

2. You enter your email and password.

3. The backend receives the login request and checks the credentials.

4. The backend may retrieve the user's information from the database.

5. The backend sends a response.

6. The frontend displays the result, such as a dashboard or an error message.

Remember: Frontend = user interface, Backend = application logic, Database = data storage.

2] Frontend vs Backend vs Database

Feature	Frontend	Backend	Database
Main purpose	Display the interface	Process application logic	Store and retrieve data
Runs mainly on	User's browser	Server	Database server or service
Examples	HTML, CSS, JavaScript, React	Java, Spring Boot	MySQL, PostgreSQL
User interaction	Direct	Through requests	Usually accessed through backend code
Example	Login page	Login verification	Saved user records

3] Client-server architecture

Client-server architecture is a system in which a client sends a request to a server, and the server processes it and returns a response.

What is a client?

A client is a device or application that requests a service.

Examples:

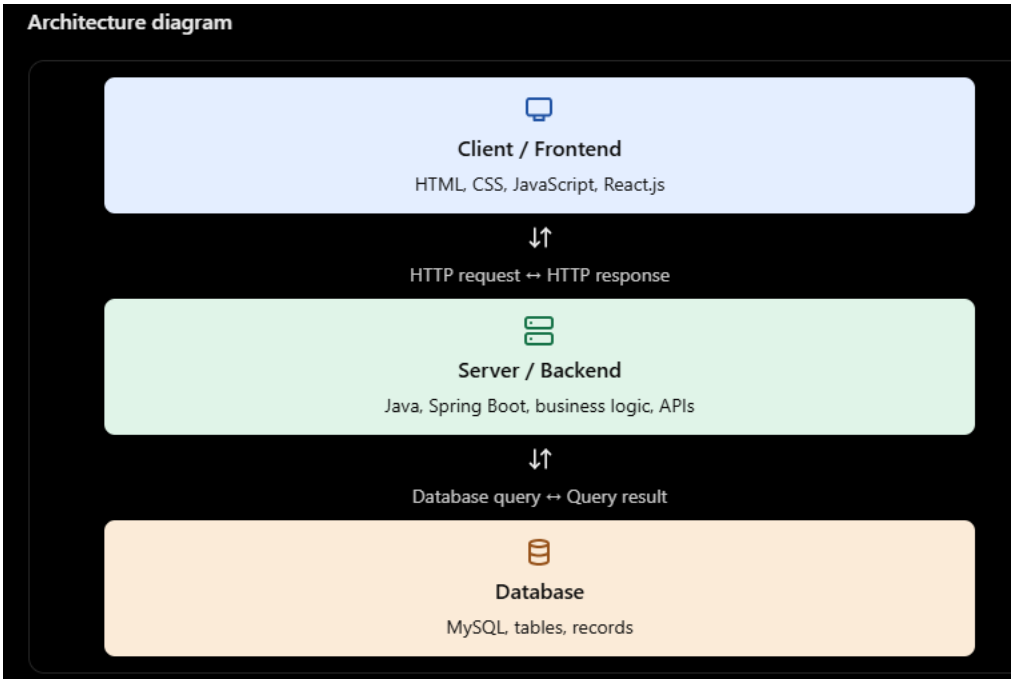
- A web browser such as Chrome.
- A mobile application.

- A desktop application.

What is a server?

A server receives requests, processes them, and sends responses.

For example, a Java Spring Boot application can act as a web server's application backend.



What is an API?

API stands for Application Programming Interface. It allows software components to communicate using defined rules.

For example, a backend might provide these API endpoints:

HTTP method	Endpoint	Purpose
GET	`/api/students`	Retrieve students
GET	`/api/students/1`	Retrieve one student
POST	`/api/students`	Create a student
PUT	`/api/students/1`	Update a student
DELETE	`/api/students/1`	Delete a student

These are example endpoints; they are not automatically available until a backend implements them.

4. How Does a Web Application Work?

Let's understand the process with a real-world example: opening a student management website.

1. User opens the website.

   The browser loads the website's HTML, CSS, JavaScript, and other resources.

2. Frontend displays the page.

   React.js, if used, can render the user interface and handle user interactions.

3. User performs an action.

   For example, the user clicks the Add Student button.

4. Frontend sends a request.

   JavaScript sends an HTTP request to the backend API, often using `fetch()` or a library such as Axios.

5. Backend processes the request.

   A Spring Boot controller receives the request, and application services perform the necessary work.

6. Database stores or retrieves information.

   The backend can use SQL and a database-access technology such as JDBC, JPA, or Hibernate.

7. Backend returns a response.

   The server commonly returns JSON data and an appropriate HTTP status code.

8. Frontend updates the screen.

   The user sees the updated student list or an appropriate error message.

Example of a request and response

A frontend sends a request to retrieve students:

```
GET /api/students
```

The backend may return JSON like this:

```
[
  {
    "id": 1,
    "name": "Asha",
    "course": "Java"
  },
  {
    "id": 2,
    "name": "Rahul",
    "course": "React"
  }
]
```

The frontend can use this response to display the student records in a table.

What is JSON?

JSON stands for JavaScript Object Notation. It is a common format for exchanging data between the frontend and backend.

5. Introduction to Java Full Stack Development

Java Full Stack Development means developing web applications using Java-based backend technologies along with frontend technologies and a database.

For a beginner following the technology list in your Day 1 PDF, the typical stack is:

Frontend

HTML, CSS, JavaScript, React.js

Backend

Java, Spring Boot

Database

SQL, for example MySQL

6] What does a Java Full Stack Developer do?

- Develop user interfaces.
- Create backend APIs using Java and Spring Boot.
- Connect applications to a database.
- Implement authentication and authorization.
- Validate user input and handle errors.

- Test application features and fix bugs.
- Maintain and improve web applications.

Example project: Task Management Application

Imagine an application where users can create and manage daily tasks.

Feature	Technology or component
Task form and dashboard	React.js
Page styling	CSS
User interactions	JavaScript
Create, update and delete APIs	Spring Boot
Application logic	Java
Store task records	MySQL
Frontend-backend communication	HTTP and JSON

When a user adds a task, React sends a request to the Spring Boot API. The backend processes the task and saves it in MySQL. The backend then returns a response, and the frontend updates the task list.